

numexpr

October 12, 2024

1 Numexpr

Sometimes the operations you want to accelerate are relatively simple Operations such as performing *elementwise* arithmetic or using *transcendental* functions.

- Elementwise such as add, subtract, divide and multiply. (e.g $C_{ij} = A_{ij} * B_{ij}$)
 - `C = A * B`
 - Dot products, matrix-vector or matrix-multiply are **not elementwise**
- Transcendental are not *algebraic* functions (i.e `exp`, `sin`)
- Numexpr is a library for the fast execution of array transformation.
- One can define complex elementwise operations on array and Numexpr will generate efficient code to execute the operations.
- Simplest way to *JIT* compile and thread code
- Remember to install it either using `conda install numexpr` or `pip install numexpr`

```
[1]: import numexpr as ne
import numpy as np
```

Numexpr provides fast multithreaded operations on array elements. Let's test it on some large arrays.

```
[2]: a = np.random.rand(1000000)
b = np.random.rand(1000000)
```

Componentwise addition is easy.

```
[3]: print(ne.evaluate("a + 1"))
print(ne.evaluate("a + b"))
```

```
[1.29776378 1.98546986 1.19112275 ... 1.11477141 1.79510228 1.02194066]
[1.08000535 1.56407786 0.36947721 ... 0.39251676 1.13972555 0.30519049]
```

We can evaluate complex expressions.

```
[4]: ne.evaluate('a*b-4.1*a > 2.5*b')
```

```
[4]: array([False, False, False, ..., False, False, False])
```

Let's compare the performance with Numpy.

```
[5]: %%timeit
a * b - 4.1 * a > 2.5 * b
```

5.98 ms ± 248 s per loop (mean ± std. dev. of 7 runs, 100 loops each)

```
[6]: %%timeit
ne.evaluate('a*b-4.1*a > 2.5*b')
```

1.59 ms ± 130 s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

Numexpr is a factor 10 faster compared to Numpy, a nice improvement with very little effort.

Let us compare some more complex operations.

```
[7]: %%timeit
np.sin(a) + np.arcsinh(a / b)
```

61.3 ms ± 1.28 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

We can compare it with Numexpr.

```
[8]: %%timeit
ne.evaluate("sin(a) + arcsinh(a/b)")
```

15.5 ms ± 2.05 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)

Numexpr is low hanging fruit when it comes to accelerating these types of problems!